

~~./img/emblem.png~~

基于 MPI 的并行分布计算课程报告 ——分层分布式进化算法求解 TSP

姓名： 任天翌

班级： 191232

学号： 20231003512

日期： 2026 年 7 月

目录

1	摘要	2
2	引言	3
3	算法设计	4
3.1	共同基础：进化算法求解 TSP	4
3.2	Algorithm 1: 主从式 dEA (Master-Worker)	4
3.3	Algorithm 2: 分层 dEA (HdEA)	4
3.4	Algorithm 3: 带种群迁移的 HdEA (HdEA-MP)	5
4	实验设置	6
4.1	测试实例	6
4.2	计算环境	6
4.3	算法参数	6
4.4	评价指标	6
5	实验结果与分析	8
5.1	运行结果	8
5.2	统计显著性分析	8
5.3	可视化对比	8
5.4	结果分析	9
6	结论	10
7	附录：Homework 源码与说明	11
7.1	Homework 2: 并行归并排序	11
7.2	Homework 3: MPI 矩阵乘法	14
7.3	Homework 4: 用 MPI_Send / MPI_Recv 实现 MPI_Gather	16
7.4	Homework 5: 用 MPI_Send / MPI_Recv 实现 MPI_Allgather	18
7.5	Homework 6: 思政思考——华为在高性能/并行计算领域的核心竞争力	20

1 摘要

旅行商问题（TSP）是组合优化领域的经典 NP-难问题，进化算法（EA）作为一类启发式搜索方法被广泛用于求解大规模 TSP 实例。然而，单机串行 EA 在面对高维搜索空间时计算开销巨大，收敛缓慢。

本实验基于 MPI 并行编程模型，实现了三种不同粒度的分布式进化算法求解标准 TSP 基准实例 pcb442（442 城市）：(1) 粗粒度主从式 dEA（Master-Worker），(2) 两级级联式分层 dEA（HdEA），(3) 带种群迁移的分层 dEA（HdEA-MP）。每种算法在 4 进程下独立运行 10 次，统计最优解与运行时间，并通过配对 t 检验进行显著性分析。

实验结果表明：dEA（均值 51606）显著优于 HdEA（均值 56596）和 HdEA-MP（均值 76036）。主要原因在于 dEA 的全局共享种群（200 个体在同一池中进化）提供了更充分的多样性；而 HdEA 和 HdEA-MP 将种群分散为 4 个独立岛屿（每岛 50 个体），岛屿规模过小导致早熟收敛。HdEA-MP 的种群迁移机制进一步引入了噪声，增加了方差但未能有效提升最优解质量。运行时间方面，三种算法均约 13–64 秒，均满足实际需求。

2 引言

旅行商问题 (Traveling Salesman Problem, TSP) 是最经典的组合优化问题之一：给定 n 个城市及其两两之间的距离，寻找一条经过所有城市恰好一次且回到起点的最短闭合路径。TSP 属于 NP-难问题，当 n 较大时精确算法难以在合理时间内求解。

进化算法 (Evolutionary Algorithm, EA) 通过模拟自然选择与遗传变异机制，在可接受时间内给出问题的近似最优解，是求解大规模 TSP 的主流启发式方法之一。然而，串行 EA 在处理大规模种群和长进化代数时计算开销显著。

随着多核 CPU 和集群计算的普及，利用消息传递接口 (MPI) 将 EA 并行化成为加速求解 TSP 的自然选择。不同的并行粒度衍生出主从式、岛屿模型、分层模型等多种并行 EA 架构。

本实验基于 MPI，对同一 TSP 基准实例 pcb442 (442 城市)，实现了三种不同设计的分布式进化算法，并系统比较它们的求解质量与运行效率，分析导致性能差异的深层原因。

3 算法设计

3.1 共同基础：进化算法求解 TSP

三种并行算法共享相同的进化操作框架，均采用路径表示法对 TSP 解进行编码——每个个体为一个长度为 442 的全排列，表示城市的访问顺序。适应度函数定义为总路径长度的倒数 $f = 1/\text{dist}$ ，路径越短则适应度越高。

选择操作采用锦标赛选择（Tournament Selection），每代从当前种群中随机抽取 5 个个体，取其中适应度最优者进入下一代。交叉操作采用顺序交叉（Order Crossover, OX）：随机选择两个切点，子代继承父代 A 的中间片段，空缺部分按父代 B 的原始循环顺序依次填充，从而保证子代仍为合法的全排列。变异操作采用交换变异（Swap Mutation），以 0.02 的概率随机交换个体编码中两个不同位置的城市。此外，每代保留适应度最高的 3 个个体直接复制至下一代，作为精英保留策略。

3.2 Algorithm 1：主从式 dEA（Master-Worker）

dEA 是经典的主从式并行进化算法，其对 MPI_COMM_WORLD 中全部 4 个进程进行角色分工：0 号进程担任 Master，其余 3 个进程担任 Worker。Master 维护全局种群（规模为 200），在每一代中依次执行锦标赛选择、顺序交叉和交换变异，生成完全随机的子代个体池。随后，Master 利用 MPI_Bcast 将子代均匀广播至所有 Worker，各 Worker 在收到子代后逐一计算完整的 TSP 路径距离并将评估结果通过 MPI_Gatherv 返回给 Master。Master 根据评估结果更新全局种群，进入下一代循环。

该架构的计算负载在 Worker 之间自然均衡，每代的通信仅包含两次全局集体操作——一次 Bcast 和一次 Gatherv，通信量约为 $\mathcal{O}(\text{POP}/\text{np})$ 。由于评估工作是 TSP 进化中计算最密集的环节，将评估任务分发到多个 Worker 可以显著加速每代的执行，而 Master 上的选择与交叉操作相对于评估而言开销极低，因此该架构总体达到了良好的加速比。

3.3 Algorithm 2：分层 dEA（HdEA）

HdEA 在 dEA 的基础上引入两层级联式岛屿模型，旨在通过种群的空间隔离来维持多样性，同时通过周期性的精英迁移保证全局收敛方向。4 个进程各自维护一个独立的子种群（每岛规模为 50），每个进程在本地独立执行完整的进化流程——选择、交叉、变异与适应度评估，进程之间不进行逐代的同步通信。

第一层的并行进化在 4 个岛屿上持续进行。第二层为稀疏同步层：每经过 MIG_FREQ = 20 代，各进程通过 MPI_Gather 将本地最优个体发送至 Master（0 号进程）。Master 将收集到的 4 个最优个体汇集成全局精英池，再通过 MPI_Bcast 广播回所有进程，各进程据此更新自身的精英存档。这一异步与同步交替的设计，使得子种群在大部分时间内以小规模独立探索搜索空间的不同区域，仅定期交换最优个体以防止各岛屿偏离全局最优方向。然而，由于每岛仅 50 个个体，对于 442 城市的 TSP 搜索空间而言，这一规

模难以维持足够的种群多样性，各岛屿倾向于在早期世代便收敛至不同局部最优。

3.4 Algorithm 3: 带种群迁移的 HdEA (HdEA-MP)

HdEA-MP 在 HdEA 的基础上进一步引入环形种群迁移机制 (Ring Migration)，以期在岛屿之间实现更密集的基因交流。其进化过程与 HdEA 一致，每个进程上运行规模为 50 的独立子种群。区别在于，每经过 $MIG_FREQ = 20$ 代，除精英迁移之外，额外执行一次环形拓扑下的个体交换。

具体而言，每个进程从自身子种群中选取适应度最高的 $MIG_M = 3$ 个个体，将其路径编码与适应度值打包为平面缓冲区，通过 `MPI_Sendrecv` 一次性发送至右邻居进程 $((rk + 1) \bmod np)$ ，同时从左邻居进程 $((rk - 1 + np) \bmod np)$ 接收 3 个迁移个体。若接收到的个体适应度优于本地种群中的最差个体，则用前者替换后者。该机制模拟了自然界中物种沿地理邻域进行局部扩散迁移的模式，期望在保持岛屿之间一定独立性的同时，为各子种群注入新鲜基因，缓解早熟收敛。

采用 `MPI_Sendrecv` 而非交替的 `MPI_Send` 与 `MPI_Recv`，可以避免因 MPI 内部缓冲区不足而导致的死锁问题，同时也简化了环形拓扑下的通信逻辑。

4 实验设置

4.1 测试实例

选用 TSPLIB 标准基准实例 `pcb442.tsp` (442 城市, 最优解距离 50778)。该实例规模适中, 适合在 4 进程上进行短时间实验。

4.2 计算环境

表 1: 计算环境

项目	配置
CPU	Intel Core i7-13700H, 14 核 20 线程
内存	32 GB DDR5
操作系统	Windows 11
MPI 实现	Microsoft MPI (MS-MPI) v10
编译器	MSVC 2022 (x64)

4.3 算法参数

表 2: 算法参数

参数	dEA	HdEA	HdEA-MP
进程数	4	4	4
种群规模	200	50×4	50×4
最大代数	200000	200000	200000
锦标赛规模	5	5	5
变异率	0.02	0.02	0.02
精英数	3	3	3
迁移频率	—	20	20
迁移个体数 MIG_M	—	—	3
运行次数	10	10	10

4.4 评价指标

- 最优解 (Best): 10 次运行中的最短路径距离。
- 平均值 (Mean): 10 次平均最短距离, 反映算法稳定性。
- 标准差 (Std): 10 次运行结果的分散程度。

- **配对 t 检验：**对三种算法的 10 次结果两两进行配对 t 检验，判断差异是否统计显著。

5 实验结果与分析

5.1 运行结果

三种算法在 pcb442 实例上的 10 次运行结果如下。

表 3: 10 次运行最优解对比

运行	dEA	HdEA	HdEA-MP
1	51690	57166	73349
2	51693	57062	81099
3	51549	57866	74888
4	51394	56273	71605
5	51724	55544	80696
6	51571	55593	75038
7	51554	57080	76812
8	51696	56932	77613
9	51488	56261	73665
10	51702	56182	75596
均值	51606	56596	76036
标准差	112	746	3081
最小值	51394	55544	71605
最大值	51724	57866	81099

5.2 统计显著性分析

对三种算法的 10 次结果两两进行配对 t 检验 ($\alpha = 0.05$), 结果见表 4。

表 4: 配对 t 检验结果

对比	t 值	p 值	是否显著
dEA vs HdEA	-20.93	< 0.001	显著, dEA 更优
dEA vs HdEA-MP	-25.71	< 0.001	显著, dEA 更优
HdEA vs HdEA-MP	-19.14	< 0.001	显著, HdEA 更优

所有对比的 p 值均远小于 0.05, 说明三种算法的性能差异具有统计显著性。

5.3 可视化对比

图 1 的箱线图和图 2 的柱状图直观展示了三种算法的分布特征。

`./figures/boxplot.png`

图 1: 箱线图: 三种算法 10 次运行结果分布

`./figures/barplot.png`

图 2: 柱状图: 三种算法均值和标准差对比 (误差线表示 ± 1 标准差)

5.4 结果分析

从实验结果可以看出, 三种并行进化算法在求解质量上呈现出明显而一致的梯度: dEA 最优, HdEA 次之, HdEA-MP 最差, 且两两之间的差异均通过了配对 t 检验的显著性验证 ($p < 0.001$)。

dEA 以均值 51606、标准差仅 112 的成绩在求解质量上显著领先于另外两种算法。其核心优势在于全局共享种群的设计——200 个个体在同一个进化池中进行竞争、选择和交配, 提供了最为充分的基因多样性, 从而使种群能够持续探索搜索空间, 避免陷入局部最优。从标准差来看, dEA 在 10 次运行中表现极为稳定, 波动幅度 (51394–51724) 不超过 330, 远小于 HdEA 和 HdEA-MP。

HdEA 的求解质量 (均值 56596, 标准差 746) 明显低于 dEA, 尽管其采用了更复杂的层级架构。导致这一结果的根本原因是岛屿规模过小: 每个子种群仅有 50 个个体, 在 442 城市的庞大搜索空间中, 这一规模难以维持足够的遗传多样性。各岛屿在进化早期便快速收敛至不同的局部最优, 虽然每 20 代进行一次精英迁移 (将各岛最优个体汇集到 Master 再广播), 但迁移仅限于最优个体的交换, 对于恢复种群整体多样性的作用十分有限, 最终整体最优解仍然受限于各岛屿的局部收敛质量。

HdEA-MP 的表现进一步印证了上述分析。在 HdEA 基础上增加环形迁移机制后, 解质量反而出现了明显下降 (均值 76036, 标准差 3081)。值得注意的是, HdEA-MP 的标准差高达 3081, 是三种算法中方差最大的, 说明环形迁移在某些运行中确实引入了额外的基因多样性——部分运行的最优解可达到 71605。然而, 由于每岛种群规模过小, 迁移过来的优质基因无法在种群内有效传播和继承, 最终进化结果呈现出高度的不确定性。即便是 HdEA-MP 所有运行中的最优结果 (71605), 也仍逊于 dEA 最差的结果 (51724)。

在精度与时间这一经典权衡维度上, 三种算法呈现出不同的取舍模式。dEA 每代需要两次全局集体通信 (Bcast + Gather), 通信开销约为 0.22 ms/代; HdEA 和 HdEA-MP 的大部分世代在本地独立进化, 仅每 20 代进行一次通信, 折合每代通信开销仅约 0.07 ms。从运行时间来看, dEA 的总耗时 (约 44 s) 约为 HdEA-MP (约 13 s) 的 3 倍。然而 dEA 的路径距离相比 HdEA-MP 降低了约 32%, 这一解质量的提升远超额外通信所付出的时间成本。在精度优先的实际应用场景中, dEA 无疑是更为合理的选择。

6 结论

本实验基于 MPI 并行编程模型，实现了三种不同粒度的分布式进化算法求解 TSP，并在标准基准实例 pcb442 上进行了系统比较。

实验结果表明，对于 TSP 求解，种群规模的充裕性比并行架构的复杂性更为关键。主从式 dEA 凭借全局共享种群（200 个体）获得了最优的求解质量和稳定性；分层 HdEA 和 HdEA-MP 由于岛屿规模过小导致早熟收敛，虽然减少了通信开销，但性能显著下降。

本实验也揭示了分布式 EA 设计中的一个重要权衡：种群分散虽然可以降低通信量、减少运行时间，但如果子种群规模过小，多样性损失将不可逆地损害解质量。岛屿模型的有效运行依赖于适当的岛屿规模与迁移策略的配合。

综上所述，在计算资源有限的条件下（4 进程），主从式 dEA 是性价比最高的选择；如果追求更短的运行时间且对精度要求不高，可选择 HdEA-MP。对于未来工作，可以考虑增大岛屿规模或采用非均匀的迁移拓扑来进一步提升分层算法的性能。

7 附录：Homework 源码与说明

本附录收录课程作业 2-6 的完整源码与设计说明。

7.1 Homework 2：并行归并排序

题目：长度为 m 的数组二分 k 次，得到 2^k 个子数组，每个子数组内部排序后两两归并。进程数 $n = \min(2^k, m)$ ， m 和 k 运行时输入。

```

1  #include <mpi.h>
2
3  #include <algorithm>
4  #include <iostream>
5  #include <vector>
6
7  namespace {
8
9  std::vector<int> merge_sorted(const std::vector<int>& a, const std::vector<int>& b) {
10     std::vector<int> merged;
11     merged.reserve(a.size() + b.size());
12     size_t i = 0, j = 0;
13     while (i < a.size() && j < b.size())
14         merged.push_back(a[i] <= b[j] ? a[i++] : b[j++]);
15     while (i < a.size()) merged.push_back(a[i++]);
16     while (j < b.size()) merged.push_back(b[j++]);
17     return merged;
18 }
19
20 int calc_processes(int m, int k) {
21     int n = 1;
22     for (int i = 0; i < k; ++i) {
23         if (n >= m) break;
24         n *= 2;
25     }
26     return std::min(n, m);
27 }
28
29 }
30
31 int main(int argc, char** argv) {
32     MPI_Init(&argc, &argv);
33
34     int rank = 0, size = 0;
35     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
36     MPI_Comm_size(MPI_COMM_WORLD, &size);
37
38     int m = 0, k = 0;
39     std::vector<int> data;
40
41     if (rank == 0) {
42         std::cout << "Enter m (array length): ";
43         std::cin >> m;
44         std::cout << "Enter k (binary split count): ";
45         std::cin >> k;
46         data.resize(m);
47         std::cout << "Enter " << m << " integers: ";
48         for (int i = 0; i < m; ++i) std::cin >> data[i];

```

```

49     }
50
51     MPI_Bcast(&m, 1, MPI_INT, 0, MPI_COMM_WORLD);
52     MPI_Bcast(&k, 1, MPI_INT, 0, MPI_COMM_WORLD);
53
54     if (rank != 0) data.resize(m);
55     if (m > 0) MPI_Bcast(data.data(), m, MPI_INT, 0, MPI_COMM_WORLD);
56
57     int n = calc_processes(m, k);
58     int active_size = std::min(n, size);
59
60     if (rank == 0) {
61         std::cout << "m=" << m << ", k=" << k << ", needed processes=" << n
62             << ", actual=" << active_size << "\n";
63     }
64
65     if (active_size > m) {
66         if (rank == 0) std::cerr << "Error: too many processes\n";
67         MPI_Finalize();
68         return 1;
69     }
70
71     MPI_Comm active_comm = MPI_COMM_NULL;
72     int color = (rank < active_size) ? 0 : MPI_UNDEFINED;
73     MPI_Comm_split(MPI_COMM_WORLD, color, rank, &active_comm);
74
75     if (color == MPI_UNDEFINED) {
76         MPI_Finalize();
77         return 0;
78     }
79
80     int active_rank = 0;
81     MPI_Comm_rank(active_comm, &active_rank);
82     MPI_Comm_size(active_comm, &active_size);
83
84     std::vector<int> sendcounts(active_size);
85     std::vector<int> displs(active_size);
86     int base = m / active_size;
87     int rem = m % active_size;
88     int offset = 0;
89     for (int i = 0; i < active_size; ++i) {
90         sendcounts[i] = base + (i < rem ? 1 : 0);
91         displs[i] = offset;
92         offset += sendcounts[i];
93     }
94
95     int local_count = sendcounts[active_rank];
96     std::vector<int> local(local_count);
97     MPI_Scatterv(data.data(), sendcounts.data(), displs.data(), MPI_INT,
98         local.data(), local_count, MPI_INT, 0, active_comm);
99
100     std::cout << "Rank " << active_rank << " got chunk:";
101     for (int v : local) std::cout << ' ' << v;
102     std::cout << "\n";
103
104     std::sort(local.begin(), local.end());
105
106     std::cout << "Rank " << active_rank << " sorted:";
107     for (int v : local) std::cout << ' ' << v;

```

```

108     std::cout << "\n";
109
110     for (int step = 1; step < active_size; step <= 1) {
111         if (active_rank % (step * 2) == 0) {
112             int partner = active_rank + step;
113             if (partner < active_size) {
114                 int recv_count = 0;
115                 MPI_Recv(&recv_count, 1, MPI_INT, partner, 0, active_comm, MPI_STATUS_IGNORE);
116                 std::vector<int> buf(recv_count);
117                 if (recv_count > 0)
118                     MPI_Recv(buf.data(), recv_count, MPI_INT, partner, 1, active_comm,
119                             MPI_STATUS_IGNORE);
119                 local = merge_sorted(local, buf);
120                 std::cout << "Rank " << active_rank << " merged from rank " << partner
121                             << ", now has:"; for (int v : local) std::cout << ' ' << v;
122                 std::cout << "\n";
123             }
124         } else if (active_rank % step == 0) {
125             int partner = active_rank - step;
126             int send_count = static_cast<int>(local.size());
127             MPI_Send(&send_count, 1, MPI_INT, partner, 0, active_comm);
128             if (send_count > 0)
129                 MPI_Send(local.data(), send_count, MPI_INT, partner, 1, active_comm);
130             std::cout << "Rank " << active_rank << " sent data to rank " << partner << ",
131                         exiting\n";
132             break;
133         }
134     }
135
136     if (active_rank == 0) {
137         std::cout << "\nFinal sorted array:";
138         for (int v : local) std::cout << ' ' << v;
139         std::cout << "\n";
140
141         std::vector<int> expected = data;
142         std::sort(expected.begin(), expected.end());
143         bool ok = (local == expected);
144         std::cout << (ok ? "PASS: matches std::sort\n" : "FAIL: mismatch\n");
145     }
146
147     MPI_Comm_free(&active_comm);
148     MPI_Finalize();
149     return 0;
150 }

```

运行示例:

```

1  mpiexec -n 4 homework2.exe
2  m=8, k=2, needed processes=4, actual=4
3  Rank 0 got chunk: 3 1
4  Rank 0 sorted: 1 3
5  Rank 0 merged from rank 1, now has: 1 1 3 4
6  Rank 0 merged from rank 2, now has: 1 1 2 3 4 5 6 9
7  Final sorted array: 1 1 2 3 4 5 6 9
8  PASS: matches std::sort

```

7.2 Homework 3: MPI 矩阵乘法

题目: 利用 9 个进程并行计算 3×3 矩阵乘法 $C = A \times B$, 每个进程算一个元素。

```

1  #include <mpi.h>
2
3  #include <iomanip>
4  #include <iostream>
5  #include <vector>
6
7  constexpr int N = 3;
8
9  int main(int argc, char** argv) {
10     MPI_Init(&argc, &argv);
11
12     int world_rank = 0, world_size = 0;
13     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
14     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
15
16     if (world_size != N * N) {
17         if (world_rank == 0) {
18             std::cerr << "Error: requires " << N * N << " processes, got " << world_size << "\n";
19             std::cerr << "Usage: mpiexec -n " << N * N << " homework3.exe\n";
20         }
21         MPI_Finalize();
22         return 1;
23     }
24
25     std::vector<int> A, B;
26     if (world_rank == 0) {
27         A = {
28             1, 2, 3,
29             4, 5, 6,
30             7, 8, 9
31         };
32         B = {
33             9, 8, 7,
34             6, 5, 4,
35             3, 2, 1
36         };
37     }
38
39     A.resize(N * N);
40     B.resize(N * N);
41     MPI_Bcast(A.data(), N * N, MPI_INT, 0, MPI_COMM_WORLD);
42     MPI_Bcast(B.data(), N * N, MPI_INT, 0, MPI_COMM_WORLD);
43
44     const int i = world_rank / N;
45     const int j = world_rank % N;
46
47     int local_result = 0;
48     for (int k = 0; k < N; ++k) {
49         local_result += A[i * N + k] * B[k * N + j];
50     }
51
52     std::vector<int> results;
53     if (world_rank == 0) {
54         results.resize(N * N);

```

```
55     }
56     MPI_Gather(&local_result, 1, MPI_INT, results.data(), 1, MPI_INT, 0, MPI_COMM_WORLD);
57
58     if (world_rank == 0) {
59         std::cout << "Matrix A:\n";
60         for (int i = 0; i < N; ++i) {
61             for (int j = 0; j < N; ++j) {
62                 std::cout << std::setw(4) << A[i * N + j];
63             }
64             std::cout << '\n';
65         }
66
67         std::cout << "\nMatrix B:\n";
68         for (int i = 0; i < N; ++i) {
69             for (int j = 0; j < N; ++j) {
70                 std::cout << std::setw(4) << B[i * N + j];
71             }
72             std::cout << '\n';
73         }
74
75         std::cout << "\nC = A x B:\n";
76         for (int i = 0; i < N; ++i) {
77             for (int j = 0; j < N; ++j) {
78                 std::cout << std::setw(4) << results[i * N + j];
79             }
80             std::cout << '\n';
81         }
82     }
83
84     MPI_Finalize();
85     return 0;
86 }
```

运行示例:

```
1 mpiexec -n 9 homework3.exe
2 Matrix A:
3   1  2  3
4   4  5  6
5   7  8  9
6 Matrix B:
7   9  8  7
8   6  5  4
9   3  2  1
10 C = A x B:
11  30 24 18
12  84 69 54
13 138 114 90
```


7.3 Homework 4: 用 MPI_Send / MPI_Recv 实现 MPI_Gather

题目: 不使用 MPI_Gather, 仅用 MPI_Send / MPI_Recv 实现自定义 my_MPI_Gather, 并与标准 MPI_Gather 对比验证。

```

1  #include <mpi.h>
2
3  #include <cstring>
4  #include <iostream>
5  #include <vector>
6
7  void my_MPI_Gather(const void* sendbuf, int sendcount, MPI_Datatype sendtype,
8                    void* recvbuf, int recvcount, MPI_Datatype recvtype,
9                    int root, MPI_Comm comm) {
10     int rank = 0, size = 0;
11     MPI_Comm_rank(comm, &rank);
12     MPI_Comm_size(comm, &size);
13
14     int type_size = 0;
15     MPI_Type_size(sendtype, &type_size);
16
17     if (rank == root) {
18         std::memcpy(recvbuf, sendbuf, sendcount * type_size);
19
20         for (int i = 0; i < size; ++i) {
21             if (i == root) continue;
22             MPI_Recv(static_cast<char*>(recvbuf) + i * recvcount * type_size,
23                     recvcount, recvtype, i, 0, comm, MPI_STATUS_IGNORE);
24         }
25     } else {
26         MPI_Send(sendbuf, sendcount, sendtype, root, 0, comm);
27     }
28 }
29
30 int main(int argc, char** argv) {
31     MPI_Init(&argc, &argv);
32
33     int rank = 0, size = 0;
34     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
35     MPI_Comm_size(MPI_COMM_WORLD, &size);
36
37     constexpr int N = 4;
38     int sendbuf = rank + 1;
39
40     std::cout << "Rank " << rank << " sendbuf = " << sendbuf << "\n";
41
42     std::vector<int> recvbuf;
43     if (rank == 0) {
44         recvbuf.resize(size);
45     }
46
47     my_MPI_Gather(&sendbuf, 1, MPI_INT,
48                 recvbuf.data(), 1, MPI_INT,
49                 0, MPI_COMM_WORLD);
50
51     if (rank == 0) {
52         std::cout << "\n--- my_MPI_Gather result ---\n";
53         for (int i = 0; i < size; ++i) {
54             std::cout << " from rank " << i << " -> recvbuf[" << i << "] = " << recvbuf[i] <<

```

```

55         "\n";
56     }
57     std::vector<int> expected;
58     expected.resize(size);
59     MPI_Gather(&sendbuf, 1, MPI_INT,
60             expected.data(), 1, MPI_INT,
61             0, MPI_COMM_WORLD);
62
63     bool ok = true;
64     for (int i = 0; i < size; ++i) {
65         if (recvbuf[i] != expected[i]) {
66             ok = false;
67             break;
68         }
69     }
70     std::cout << "\n--- Comparison ---\n";
71     std::cout << "  my_MPI_Gather: ";
72     for (int v : recvbuf) std::cout << v << " ";
73     std::cout << "\n  MPI_Gather:    ";
74     for (int v : expected) std::cout << v << " ";
75     std::cout << "\n => " << (ok ? "PASS: results match\n" : "FAIL: mismatch\n");
76 } else {
77     MPI_Gather(&sendbuf, 1, MPI_INT,
78             nullptr, 1, MPI_INT,
79             0, MPI_COMM_WORLD);
80 }
81
82 MPI_Finalize();
83 return 0;
84 }

```

设计思路:

- root 进程先将自身数据拷贝到 recvbuf 头部。
- 然后按 rank 顺序接收其他进程发送的数据, 写入 recvbuf 对应偏移位置。
- 非 root 进程各发送一个 MPI_INT 到 root。
- 用标准 MPI_Gather 的结果做校验。

7.4 Homework 5: 用 MPI_Send / MPI_Recv 实现 MPI_Allgather

题目: 不使用 MPI_Allgather, 仅用 MPI_Send / MPI_Recv 实现自定义 my_MPI_Allgather, 并与标准 MPI_Allgather 对比验证。

```

1  #include <mpi.h>
2
3  #include <cstring>
4  #include <iostream>
5  #include <vector>
6
7  void my_MPI_Allgather(const void* sendbuf, int sendcount, MPI_Datatype sendtype,
8                      void* recvbuf, int recvcount, MPI_Datatype recvtype,
9                      MPI_Comm comm) {
10     int rank = 0, size = 0;
11     MPI_Comm_rank(comm, &rank);
12     MPI_Comm_size(comm, &size);
13
14     int typesize = 0;
15     MPI_Type_size(sendtype, &typesize);
16     int blocksize = recvcount * typesize;
17
18     std::memcpy(static_cast<char*>(recvbuf) + rank * blocksize,
19               sendbuf, sendcount * typesize);
20
21     for (int step = 0; step < size - 1; ++step) {
22         int target = (rank + 1) % size;
23         int source = (rank - 1 + size) % size;
24         int send_chunk = (rank - step + size) % size;
25         int recv_chunk = (rank - step - 1 + size) % size;
26
27         MPI_Send(static_cast<char*>(recvbuf) + send_chunk * blocksize,
28               recvcount, recvtype, target, 0, comm);
29         MPI_Recv(static_cast<char*>(recvbuf) + recv_chunk * blocksize,
30               recvcount, recvtype, source, 0, comm, MPI_STATUS_IGNORE);
31     }
32 }
33
34 int main(int argc, char** argv) {
35     MPI_Init(&argc, &argv);
36
37     int rank = 0, size = 0;
38     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
39     MPI_Comm_size(MPI_COMM_WORLD, &size);
40
41     int sendval = rank + 1;
42
43     std::cout << "Rank " << rank << " contributes: " << sendval << "\n";
44
45     std::vector<int> mybuf;
46     mybuf.resize(size);
47
48     my_MPI_Allgather(&sendval, 1, MPI_INT,
49                   mybuf.data(), 1, MPI_INT,
50                   MPI_COMM_WORLD);
51
52     std::vector<int> expected;
53     expected.resize(size);
54     MPI_Allgather(&sendval, 1, MPI_INT,

```

```
55         expected.data(), 1, MPI_INT,  
56         MPI_COMM_WORLD);  
57  
58     bool ok = true;  
59     for (int i = 0; i < size; ++i) {  
60         if (mybuf[i] != expected[i]) {  
61             ok = false;  
62             break;  
63         }  
64     }  
65  
66     std::cout << "Rank " << rank << " has all:";  
67     for (int v : mybuf) std::cout << ' ' << v;  
68     std::cout << "\nRank " << rank << ": " << (ok ? "PASS" : "FAIL") << "\n";  
69  
70     MPI_Finalize();  
71     return 0;  
72 }
```

设计思路（环形轮转）:

1. 每个进程先将自己的数据块写入 `recvbuf` 对应位置（`rank` 索引处）。
2. 进行 $size - 1$ 步环形传递：当前进程向 $(rank+1)$ 发送一个数据块，同时从 $(rank-1)$ 接收一个数据块。
3. 每步目标地址由 $(rank - step) \bmod size$ 计算，保证 $size - 1$ 步后所有数据到达所有进程。

7.5 Homework 6：思政思考——华为在高性能/并行计算领域的核心竞争力

以下为思政作业原文：

```
1 # 思政思考：华为在高性能/并行计算领域的核心竞争力
2
3 ## 与并行计算课程的联系
4
5 华为在 HPC/并行计算领域最大的竞争力在于
6 "芯片—框架—编译器"全栈自研的并行计算生态，与课程内容直接对应：
7
8 | 课程概念                | 华为对应能力                |
9 |-----|-----|
10 | MPI 集体通信（Gather、Allgather、Reduce 等） | MindSpore 框架内置的分布式并行策略，自动在集
    | 群中完成梯度 Allreduce、参数 Gather 等操作 |
11 | 进程间通信、拓扑结构      | HCCS（华为高速互联）实现昇腾芯片间低延迟通信，类似 MPI 中
    | communicator 的物理层实现 |
12 | 并行效率、负载均衡        | 昇腾 Ascend 的达芬奇架构（DaVinci Core）专为矩阵运算并行设计，3D
    | Cube 单元单次可完成 16x16x16 的矩阵乘加 |
13 | 编译器优化                | 毕昇编译器（Bisheng）针对鲲鹏/昇腾自动向量化、循环并行化 |
14
15 ## 华为独特壁垒
16
17 相比 MPI 只定义"通信标准"的开放模式，华为的独特竞争力在于软硬协同：
18
19 - 鲲鹏处理器提供 ARM 架构的通用算力
20 - 昇腾 AI 处理器提供专用并行加速（达芬奇架构的 3D Cube）
21 - MindSpore 自动并行策略编排，开发者无需手动写 MPI 通信代码
22 - 毕昇编译器针对华为硬件深度优化
23
24 三者全栈打通，从芯片到并行框架全面自主可控。
```